

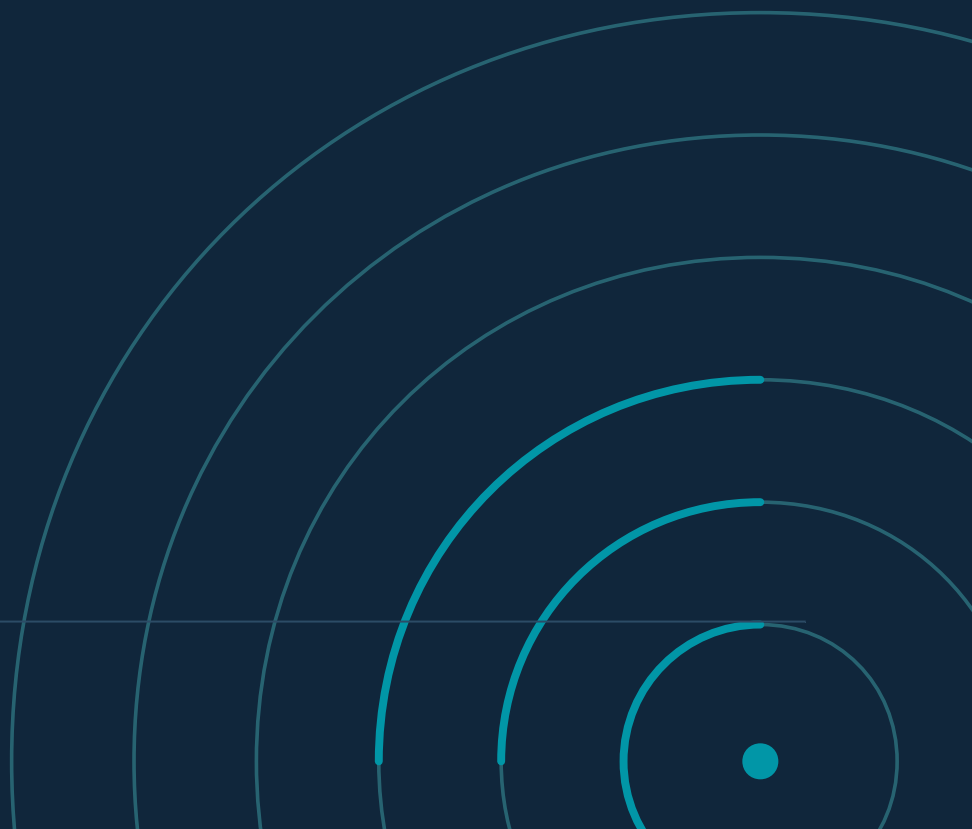


PRODUCT REQUIREMENTS DOCUMENT

SGW Resilience Platform

The whole product in one sentence: bring the scattered facts together, rank the risks with plain reasons, keep people in charge of every decision — and build the smallest honest test first, before spending big.

NINE SECTIONS · ONE PAGE EACH





Q	—	PRD at a Glance	3
📄	—	Executive Summary	4
🎯	01	Problem Definition & Business Context What exactly is broken, and why does it cost money?	5
⚖️	02	Key Assumptions & Unknowns The case leaves things unsaid , what did we have to guess?	6
👤	03	Target Users & Pain Points Who will use this, and where does their day break down?	7
📋	04	Functional & Non-Functional Requirements What must the system do, and how well?	8
⚙️	05	Proposed AI Capabilities What does the AI actually do?	9
🗃️	06	High-Level Architecture & Integrations How is the system built, and how does it connect to SGW's tools?	10
📊	07	Data Requirements & Dependencies What data is needed, and what is wrong with data like this?	11
🛡️	08	Security, Governance & Human-in-the-Loop Who is responsible, and what stops the AI from causing harm?	12
📈	09	Success Metrics, MVP Scope & Delivery Priorities How do we measure success, and what gets built first?	13

Every section answers one requirement from the case, on a single page.



PRD at a Glance

Background for a first-time reader: SGW is a utility company serving more than 8 million people with power and water, in a region hit by hurricanes, floods, heatwaves and wildfires. This document plans a software platform that helps SGW's people make faster, fact-based decisions during storms. The table below summarizes all nine sections of the plan.

#	Section	Question it answers	The answer, in short
1	Problem Definition & Business Context	What exactly is broken, and why does it cost money?	SGW's storm decisions are made by capable people — but the facts they need sit in four computer systems that do not share data. So decisions run on memory and gut feeling. Slow, blind decisions mean longer power cuts, penalty fees, expensive emergency crews, and hard questions from regulators. The platform brings all the facts into one place and shows a ranked list of risks with plain reasons. People still make every decision — the computer only helps.
2	Key Assumptions & Unknowns	The case leaves things unsaid, what did we have to guess?	We wrote down all eight guesses we had to make, each with its reason and with what happens if it is wrong. The three most dangerous unproven guesses: (1) SGW's data can actually be pulled out of its systems; (2) seeing everything in one place really changes the crew decision; (3) operators will act on rankings made by a computer. The first small version of the product exists to test exactly these three guesses — before any big money is spent.
3	Target Users & Pain Points	Who will use this, and where does their day break down?	Four people: the operations manager (plans where crews wait before the storm), the dispatcher (chooses what gets repaired first during the storm), the field crew lead (does the repairs), and the executive (answers to regulators and press). We wrote the storm as a six-step story and marked the exact moments where today's way of working breaks — for example: the forecast changes, and hours of manual planning start again from zero.
4	Functional & Non-Functional Requirements	What must the system do, and how well?	Eight things it must do — for example: rank all assets by risk, let a person accept or reject every suggestion, dismiss a false alarm in one click, and keep a record of every decision. Five rules for how well it must work — for example: it must keep working even when the storm cuts its data feeds, and every number on screen must show where it came from and how old it is. Next to each requirement, we name the finding that created it — nothing is there without a reason.
5	Proposed AI Capabilities	What does the AI actually do?	Three abilities, one for each problem SGW named: (1) a risk forecast for every asset for the next 72 hours; (2) an early warning when sensor readings start looking abnormal; (3) an automatically drafted summary for leadership, checked by a person before it goes out. Every ability only advises — a person always makes the decision. Two AI ideas from the case (analyzing drone photos, automatic route planning) were deliberately not used, each with a written reason.
6	High-Level Architecture & Integrations	How is the system built, and how does it connect to SGW's tools?	Five layers: collect the data from SGW's four existing systems → store it in one place → the AI reads it and makes suggestions → a server applies sign-in rules and the human-approval step → four simple screens, one per user. One safety wall above all: the platform can only READ from the systems that physically control the grid and water — it can never send them commands. Every connection has a backup plan in case a modern interface does not exist.
7	Data Requirements & Dependencies	What data is needed, and what is wrong with data like this?	Six data sources are needed, each with a backup plan. We also opened real public data files of the same kinds and found seven real problems — for example: 97% of wind-gust measurements missing, and outage records with impossible numbers. Each problem has a design answer on the page. The one thing only SGW can provide: its own history of which assets failed in past storms — checking that history is a first-week task.
8	Security, Governance & Human-in-the-Loop	Who is responsible, and what stops the AI from causing harm?	Four questions answered in writing before launch: who is responsible (a named person, not a department), what could go wrong and who could get hurt (four dangers, each with a built-in protection — including fairness between rich and poor neighborhoods), how it is tested before launch, and who watches it after launch. Plus an off-switch: SGW can turn the AI off at any moment and keep working manually with all data still visible.
9	Success Metrics, MVP Scope & Delivery Priorities	How do we measure success, and what gets built first?	Success is measured, not felt. The main number: how many real failures the system flagged in advance — starting target, at least 7 out of 10. The first version built (the MVP) is deliberately small: one storm scenario, end to end, tested with real operators — because its job is to test the three dangerous guesses from Section 2. Growth happens in three steps, and each step must pass a written test before the next one is allowed to start.

The whole product in one sentence: bring the scattered facts together, rank the risks with plain reasons, keep people in charge of every decision — and build the smallest honest test first, before spending big.



What I Did, and How

The case document leaves many things unsaid — on purpose. My answer to that was a fixed working method. I moved through the problem in ordered steps. In every step, I used one established, published method, so that every choice in this document has a source behind it. When the case was silent, I did not stop, and I did not guess quietly: every guess is written down, with its reason and with what happens if it is wrong. When a number was needed, I set a starting target and labeled it as a guess to be confirmed with the client. And every requirement in this document can be traced back to a finding — nothing comes from imagination.

The writing follows one rule everywhere: plain language first, so anyone can read it. The official name of each method appears in brackets, for the technical reader.

What I did for each section — and what came out of it:

#	Section	What I did, step by step	What came out of it
1	Problem Definition	I read the case once and sorted every sentence into three boxes: facts, unknowns, and things we must deliver (method: Volere). Then I wrote the problem as one decision and checked its six parts, one by one (method: Decision Quality — R. Howard, Stanford). Finally I cut the problem into pieces and asked for each: can experts plan this on paper, or can only a real test answer it? (methods: Four Risks — M. Cagan; Cynefin — Kurtz & Snowden, 2003)	A precise diagnosis: only two parts of the decision are broken — scattered information, and thinking without facts. And a clear split: what this document plans fully, and what the first small version must test.
2	Assumptions & Unknowns	For every unknown, I wrote the guess in one fixed sentence: “We assume X, because Y. If wrong, Z happens.” Then I sorted all guesses with two questions: can this kill the project if wrong? And: do we have real proof? (method: Assumption Mapping — Bland & Osterwalder, 2019)	Eight written guesses. Three of them are both dangerous and unproven — they decide what must be tested first, before big money is spent.
3	Users & Pain Points	I found the users with a rule, not by imagination: for each broken decision, who makes it, who acts on it, who reports on it? Then I described each person's work in one fixed sentence: what they do, how long it takes, and when it breaks (method: Contextual Inquiry — Beyer & Holtzblatt). The case is fictional, so every row is honestly labeled as a reasoned guess.	Four users. The storm written as a six-step story, with the exact breaking moments marked — each one a thing the product must fix.
4	Requirements	I converted findings into features with a strict rule: every requirement must come from an earlier finding, and every finding must get a requirement — nothing invented, nothing forgotten. Each feature is one sentence: [person] must be able to [do something], so that [a result]. Each quality rule has a number and a plan for failure — never empty promises like “it never fails”.	8 features and 5 quality rules. Next to each one, a note saying which finding created it — so anyone can ask “why is this here?” and get an answer.
5	AI Capabilities	An AI idea was allowed in only if it clearly improves a named person's decision — a one-page test with six questions (method: Machine Learning Canvas — L. Dorard). I chose each technology by reading published research and comparisons, not by fashion. Ideas that failed the test were rejected in writing, with reasons.	Three AI abilities, one per problem SGW named — each with a plan to prove it before launch and to watch it after launch. Two ideas from the case declined in writing.
6	Architecture	I did not invent a design. I used the standard industry pattern for systems of this kind: collect the data → store it → analyze it → serve it → show it. On top, I applied the utility industry's safety practice: an analysis system may only READ from the systems that control the grid — never command them.	Five clear layers. A one-way safety wall toward the control systems. A backup plan for every connection to SGW's existing tools.
7	Data & Quality	I did not guess what goes wrong with this kind of data. I downloaded real public data files of the same kinds — US outage records, weather station files, water sensor data — opened them, and measured their problems. Those measured problems, not imagined ones, went into the plan.	The list of needed data with a backup for each. Seven real, verified data problems, each with a design answer. And the one thing only SGW can provide: its own failure history.
8	Governance & Security	I answered, in writing, the four questions of the US government's framework for trustworthy AI (NIST AI Risk Management Framework, 2023): who is responsible? What could go wrong, and who could get hurt? How is it tested before launch? Who watches it after launch?	A named owner. Four dangers, each with a built-in protection. A test plan. A watch plan — and an off-switch that returns SGW to manual work at any moment.
9	Metrics & First Version	I priced the two possible mistakes: missing a real failure costs roughly a thousand times more than a false alarm (method: cost-sensitive learning — C. Elkan, 2001). That price — not habit — sets the success numbers. The first version to build is simply the cheapest honest test of the dangerous guesses from Section 2.	The main success number: real failures flagged in advance. A deliberately small first version. A three-step growth plan, with a written test between every step.

How to read this document: on the following pages, I explain each of the nine sections individually — one page per section. Every page starts with the exact requirement it answers (quoted word for word), then a short note on how I produced that section, and then the content itself, in plain language.



Case requirement (verbatim): “Clearly articulate the operational problem and why it matters to SGW.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. To find this problem, I used only the facts written in the case document. I did not invent anything. I used three methods. Method 1: I sorted every sentence of the case into three groups — facts, unknowns, and required deliverables (Volere method). Method 2: I wrote the problem as a decision, because this product must support a decision. Then I checked the six parts of a good decision, one by one (Decision Quality method, R. Howard, Stanford). Method 3: I cut the problem into small pieces. For each piece I asked: can experts plan this on paper, or do we need to build and test it? (Four Risks, Cagan; Cynefin, Kurtz & Snowden 2003). My reasoning chain was: facts → the decision → the broken parts of that decision → the product’s job.

What SGW faces. SGW gives power and water to more than 8 million people. The coastal areas get hurricanes and floods. The inland areas get heatwaves and wildfires. The information SGW needs already exists. But it is stored in four separate computer systems, and these systems do not share data with each other: asset maps (GIS), maintenance records, weather feeds, and field-crew tools. At the same time, the business is under pressure: costs are going up, power cuts are more frequent, insurance is more expensive, and regulators demand better storm preparation.

The problem in one sentence: *when a storm is 1 to 3 days away, the operations manager must decide where to send the repair crews — to the coast, inland, split between both, or hire expensive extra crews from other companies.*

What is broken in this decision — and what is not. A good decision needs six parts. We checked each part for SGW:

Part of a good decision	Works?	What we found
A clear question	Yes	Everyone knows what must be decided
Real options to choose from	Yes	Coast, inland, split, or extra crews — real, different options
The right information	NO	The facts sit in 4 systems that do not connect
Clear goals	Yes	Bring power back fast; keep people safe
Clear thinking based on facts	NO	Facts are missing, so the choice is based on personal experience
Someone who takes action	Yes	The manager decides, and the crews move

What this tells us. SGW’s people do their job well. Four of the six parts are already working. Only two parts are broken. First, the **information**: the facts are spread across four systems. Second, the **thinking**: because facts are missing, decisions are based on personal experience, not on data. The same two problems appear again during the storm, when the dispatcher must choose which broken thing to repair first, using only radio calls, alarms, and a whiteboard. So the **platform** must do only two things: **(1) bring all the information into one place**, and **(2) turn it into a ranked list of risks, with a simple reason next to each one**. People still make the final decision. The computer only helps. This is why we call it decision support, not automation.

Why it matters. Slow decisions based on missing facts cost money and hurt people. Power stays off longer. SGW pays penalty fees. Extra crews are very expensive. Insurance prices go up. Regulators ask questions that SGW cannot answer with numbers. Fixing the two broken parts helps with all four business pressures at the same time.

What we can plan now — and what we must test first. Some pieces can be fully planned on paper by experts. Other pieces can only be answered by building a test version and showing it to real operators:

Piece of the problem	Can experts plan it on paper?	So we will...
Connect the 4 systems; predict storm damage for each asset	Yes — an engineering task	Plan it fully in this document
Will one combined view really change the crew decision?	No — only a test can answer this	Test it with the prototype
Will operators trust risks ranked by a computer, during a storm?	No — only a test can answer this	Test it with the prototype, then watch real operators
Are the benefits bigger than the costs? Will regulators accept it?	Yes — analysis and review	Business case + compliance check
Using any tool during the chaos of a live storm	No — it is a crisis situation	Design rule: keep every screen very simple



Case requirement (verbatim): “State all assumptions explicitly and explain how they shape the solution.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. The case document is incomplete on purpose. So I collected every guess I had to make in the earlier work: the unknowns from the case sort, the guessed workflows, and every risk question I could not answer with a clear yes. I wrote each guess in one fixed form: “**We assume X, because Y. If this is wrong, then Z happens.**” The “**if wrong**” part is important, a guess without a consequence is useless. Then I sorted every guess with two questions: (1) Can the project die if this guess is wrong? (2) Do we have real proof for it? Guesses that are dangerous AND unproven go to the top of the list. They must be tested first, and the cheapest thing that tests them IS the **MVP**. (Method: Assumption Mapping, Bland & Osterwalder, 2019.)

Why this section exists. We do not hide our guesses. We write every guess down, we explain why we believe it, and we say what happens if it is wrong. This way, the client can correct us fast, and the plan can change safely.

The eight assumptions. Each one has three parts: **the guess, the reason, and the consequence if wrong**. The priority column shows the sorting result:

ID	We assume...	Because...	If it is wrong...	Priority
A1	Asset data (location, age, condition) can be pulled out of the GIS and maintenance systems	The case says these systems exist	We start with weather data and public records instead; fixing the data becomes Phase 1	TEST FIRST
A2	A combined, ranked risk view will really change the crew decision — the biggest guess of all	Section 1 showed the decision is missing information and clear thinking	The whole product idea fails. This is why the prototype tests exactly this	TEST FIRST
A3	Operators will act on computer rankings if every ranking shows its reasons	Research supports it in general; we have no proof from SGW	We redesign: operators control the filters, and the AI only adds notes	TEST FIRST
A5	Asset condition data is partly old and partly estimated	Utilities normally inspect assets every few years	Already handled: every value on screen shows its source and its age	Build on it
A8	No law forbids AI advice when a human keeps the final authority	This is the normal rule today	We add stricter oversight; the design stays the same	Build on it
A4	Weather data can be mapped down to each single asset	Forecast grids can be interpolated	Extra data engineering work — not fatal	Note it
A6	Storm planning today takes hours and restarts when the forecast changes	Reasoned guess from the workflow study	The story changes, the product idea does not	Note it
A7	Records of past storms and failures exist, enough to test the model on history	Utilities usually keep such records	We test with expert review and simulated storms instead	Watch it

Why only three are “test first”. A guess goes to the top only if BOTH answers are bad: it can kill the project, and we have no proof. A1, A2 and A3 are the only ones that fail both tests. This short list is a good thing. If ten things are “test first”, then nothing is really first. A short list makes a sharp MVP: the prototype is the cheapest thing that tests A2 and A3, and nothing more. A7 is watched: if testing on past storms becomes our main proof plan, A7 moves to the top.

Open questions for the client. How often do assets fail, and what does one failure cost? Who makes which decision today? How does coordination work now? How good and how fresh is the data? What is the budget? What are the IT and security rules?



Case requirement (verbatim): “Define the operational workflows and user groups most affected.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. I did not brainstorm “who might use this.” The users come from the two broken decisions in Section 1, using a fixed rule: for each decision, find who makes it, who acts on it, and who must report on it. That gives exactly four roles. For each role, I filled one fixed sentence: [Role] does [steps] → it takes [time] → it breaks when [condition]. This sentence form comes from Contextual Inquiry (Beyer & Holtzblatt), the method of watching people do real work. The case is fictional, so there was no control room to watch. I filled the sentences by reasoning instead, from three inputs: the case facts (four disconnected systems force manual collection), normal utility practice (control rooms, radio, whiteboards), and logic (a hand-made plan must be re-made by hand when the forecast changes). The time numbers are labeled guesses (assumption A6). In a real project, the first two weeks replace every guess with real observation.

Why this section exists. A tool is only useful if it fits the real work of real people. This section describes the four people the platform serves, what they do today, and the exact moment their current way of working breaks. Each breaking moment is a pain point, and each pain point is something the platform must fix.

The four users. Every row below is a reasoned guess, to be replaced by real observation in Phase 0:

User	What they do today	How long it takes	It breaks when...
Operations manager (before the storm)	Collects weather emails, map exports, and phone calls from maintenance. Builds the crew plan by hand.	Hours, spread over a day	The forecast changes — and all the collection work starts again
Dispatcher (during the storm)	Follows radio calls, alarms, and a whiteboard. Builds one picture of the damage in their head.	About 30 minutes for one full picture	Damage reports arrive faster than anyone can write them down
Field crew lead (in the field)	Gets jobs by phone or radio. Reports status back by voice.	Constant back-and-forth	Radio and phone signal degrade in the storm — crews drive to the same place twice
Executive (during the event)	Calls the operations manager and asks for a spoken update.	Minutes per call — but old news within an hour	A regulator or a journalist asks for numbers that nobody has ready

The whole storm, end to end — how the four workflows connect today. This is the full operational workflow as it runs now. The word **BREAK** marks where it fails:

Step	What happens today	Problem
1. Storm is 48h away	The operations manager starts collecting: weather emails, map exports, phone calls to maintenance. Builds the crew plan by hand.	Takes hours
2. Forecast changes	All the collection work starts again from zero.	BREAK
3. Storm arrives	Alarms and radio calls flood in. The dispatcher builds one damage picture in their head from radio, alarms, and a whiteboard.	~30 min per picture
4. Repairs are assigned	The dispatcher assigns crews by radio. Crews report back by voice. When signal drops, crews drive to the same place twice.	BREAK
5. Leadership needs answers	The executive phones the manager for a spoken update. A regulator asks for numbers; nobody has them ready.	BREAK
6. After the storm	There is no clean record of what was decided, when, and why — so learning for the next storm is hard.	Silent gap

What the pain points tell us. All four breaking moments have the same two causes from Section 1: the information is scattered, and the thinking has no data to stand on. This is a good sign for the design. It means one shared foundation — all the information in one place — can serve all four people, each through their own simple view: a planning view for the manager, a live repair list for the dispatcher, a shared job list for the crews, and a numbers-ready summary for the executive.

The most important sentence for the client. These workflows are our best reasoned guesses, clearly labeled. The first two weeks of a real project are spent sitting inside the real control room, watching real dispatchers work, and replacing every guess in this table with a fact. A product built on guessed workflows fails quietly; a product built on observed workflows gets used.

Functional & Non-Functional Requirements



Case requirement (verbatim): “Specify system behavior, performance, availability, and security needs.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. Functional requirements say what the system must DO (the features). Non-functional requirements say how WELL it must work (speed, security, reliability, simplicity). I did not invent any of them. Every requirement is an earlier finding, converted into a feature: the two broken decision parts from Section 1, each “it breaks when” from Section 3, the test-first assumptions from Section 2, the mistake prices (a missed failure costs ~1000× a false alarm, so the system over-warns on purpose — Elkan, 2001), and the governance rule that humans decide. Two rules keep the list honest: (1) no requirement without a parent finding — no invention; (2) no finding without a requirement — no forgotten pains. Each functional requirement follows one fixed sentence: [Actor] must be able to [action] [object] so that [outcome]. Each non-functional requirement is a category plus a measurable statement — no impossible claims like “never fails”; instead: limits, and clear behavior when things fail.

Functional — what the system must do:

ID	Requirement	Comes from
F1	The system must join asset data from all four sources (maps, maintenance, weather, field tools) into one view per asset, so that nobody collects facts by hand anymore.	Section 1: the missing-information gap
F2	The operations manager must be able to see all assets ranked by risk for the next 72 hours, and the ranking must update itself when the forecast updates, so that a forecast change adjusts the plan instead of restarting it.	Section 1: the weak-thinking gap · Section 3: the forecast-change break
F3	Every user must be able to see the reasons behind each risk rank, written in plain words (e.g., “old transformer + flood zone + strong wind expected”), so that people can judge and trust the ranking.	Section 2, assumption A3: trust needs visible reasons
F4	The operator must be able to accept, change, or reject every recommendation — and the system must never send crews anywhere by itself — so that a human always makes the final decision.	Section 8 (governance): humans keep the final decision
F5	The dispatcher must be able to dismiss a false alarm with one click, so that the many cheap alarms (which exist by design) do not slow down real work.	Section 9 (error costs): over-warning is deliberate
F6	The dispatcher and the field crews must be able to see one shared, live list of damage and repair jobs, so that nothing is missed and no crew drives to the same place twice.	Section 3: the report-flood break + double crew trips
F7	The executive must be able to get a summary with numbers (outages, crews, repair estimates), with a time stamp, at any moment, so that regulators and press get facts instead of guesses.	Section 3: the no-numbers-ready break
F8	The system must save every recommendation and every human decision with a time stamp, so that SGW can learn from each storm and prove how decisions were made.	Section 3, step 6: no record of decisions today

Non-functional — how well it must work:

ID	Category	Requirement	Comes from
NF1	Availability	The platform must survive the storm it serves. If data feeds fail, it must keep working and show the last saved picture, clearly marked as old.	Section 1: the live-storm crisis row
NF2	Usability	A stressed operator must be able to do any critical action in two clicks or less, without a manual.	Section 1: the live-storm crisis row
NF3	Performance	The risk list must refresh within minutes of a forecast update; the live repair list must show new reports within seconds.	The case requirement itself: performance
NF4	Security	Only signed-in users get access, and each person sees only what their job needs. Data is encrypted. The platform may only READ from the systems that physically control the grid — it can never send commands to them. Every access is logged. The official US security rules for power companies (called NERC CIP) will be checked in a compliance review.	The case requirement itself: security + utility practice
NF5	Honesty of data	Every number on screen must show its source and its age; estimated values must look clearly different from measured ones.	Section 2, assumption A5: partly old, estimated data

A note on the numbers. “Within minutes” and “two clicks” are targets written as assumptions. The real limits are set with the client at the start of the project, after we measure how the work happens today. A precise-sounding number without a basis is just an impossible claim in disguise.



Case requirement (verbatim): “Describe expected AI behaviors and how they improve decision-making.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. I did not brainstorm AI ideas. An AI ability is allowed into this product only if it clearly improves one of the decisions named in Sections 1 and 3. The rule behind this: a prediction that changes no decision is worthless (Machine Learning Canvas method, L. Dorard). Every admitted ability must answer the same six questions: what data goes in, what it predicts, which decision it feeds, what gets better, how we prove it works before launch, and how we keep checking it after launch. If any answer is empty, the ability is not built. The choice of technology inside each ability comes from published research, not from fashion — the key studies are named below in plain words. Finally, the case document itself suggests AI ideas; each suggestion is either used here or rejected with a written reason at the bottom.

Why exactly three abilities — and where each one comes from. The case names three pain points, so there are three abilities — one for each. Each ability was found by the same chain: the pain point → the person and the broken moment behind it (Sections 1 and 3) → the output that would repair it:

The case's pain point (their words)	The person and the broken moment	The ability that answers it
“Proactively assess infrastructure risks”	The operations manager, placing crews before the storm using scattered facts and personal judgment	1 · Risk forecast per asset
“Coordinate emergency response activities”	The dispatcher, drowning in radio calls and alarms — about 30 minutes to build one full picture	2 · Early warning on sensors
“Real-time situational awareness for leadership”	The executive, on the phone, while a regulator asks for numbers that nobody has ready	3 · Automatic summary writer

Three abilities — one for each of SGW's three problems. These are Canvases 1, 2 and 3 from the discovery work (Stage 5 in the discovery source-of-truth document). Each column answers the same six questions:

Question	1 · Risk forecast per asset — from Canvas 1 (technical name: predictive machine learning)	2 · Early warning on sensors — from Canvas 2 (technical name: anomaly detection)	3 · Automatic summary writer — from Canvas 3 (technical name: language model, LLM)
What data goes in?	Asset maps + maintenance history + weather forecast	Live sensor readings and incoming damage reports	The numbers the platform already holds (outages, crews, repairs)
What does it predict or produce?	A risk score for each asset for the next 72 hours, with a confidence level	A flag: “this pattern looks abnormal” — with the context around it	A short written summary of the situation, with a time stamp
Which decision does it improve?	Where to place the crews before the storm (the manager's decision)	What to repair first during the storm (the dispatcher's decision)	What to spend and what to tell regulators and press (the executive's decision)
What gets better?	Plan in minutes instead of hours; plan adjusts when the forecast changes	Problems seen earlier; repairs ordered by facts, not by loudest voice	Numbers ready in minutes, always current, always traceable
How do we prove it before launch?	Replay past storms: did the damaged assets rank high on the list?	Replay past failures: how early would the flag have come?	Compare its summaries with summaries people wrote in past events
How do we watch it after launch?	After every storm, compare predictions with reality; retrain when quality drops	Count false alarms every week; watch what operators dismiss	Count corrections people make; check every fact against the database

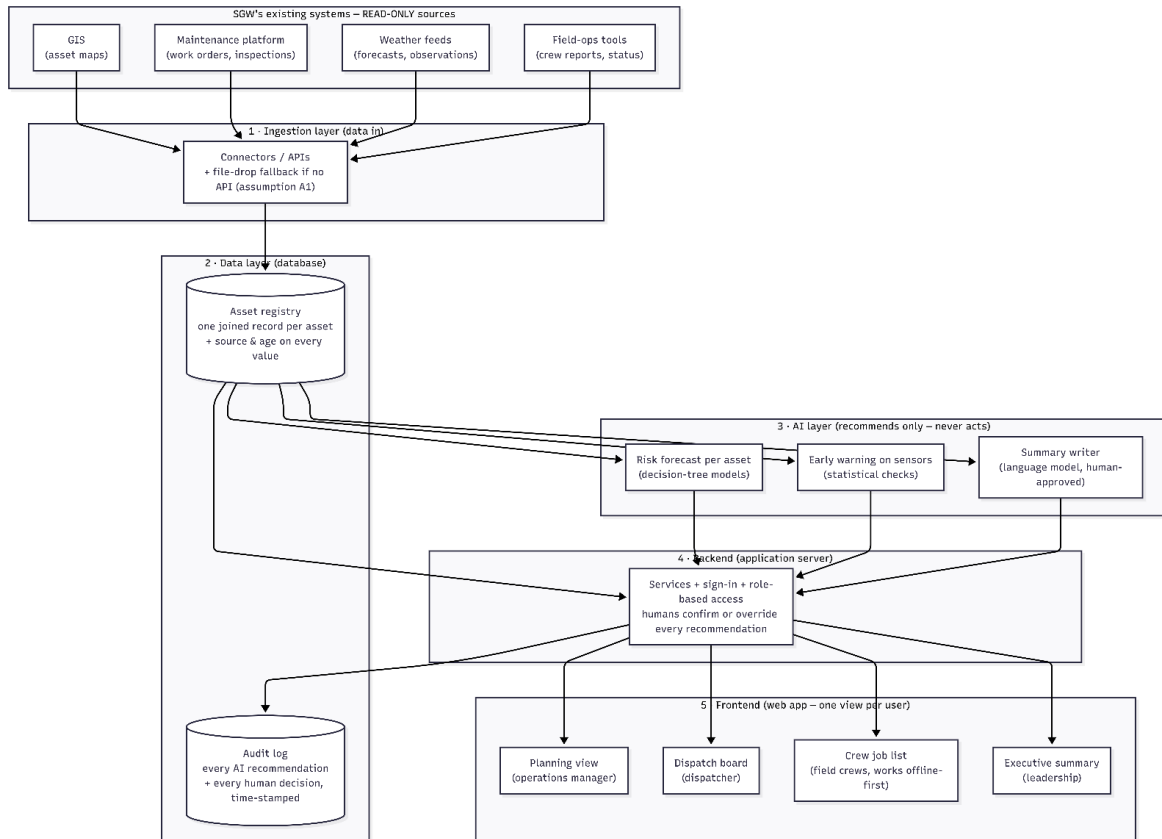
Why these technologies — in plain words, from research. Ability 1 uses decision-tree models: many simple if-then rules combined. A major study (NeurIPS 2022) showed these models beat modern deep learning on exactly this kind of data — tables of asset facts — and they can show their reasons, which the trust requirement (F3) needs. Storm-damage research has used this model family since 2014. Its known weakness: it cannot judge storms stronger than any it has seen — one more reason a human keeps the final decision. **Ability 2** starts with simple statistical checks (learn what normal looks like, flag what is not); the largest independent comparison (71 methods, 2022) found simple methods work as well as complex ones here — we upgrade only if our own tests prove the need. **Ability 3** uses a language model (the technology behind tools like ChatGPT) — but with strict rules: it may only re-write numbers the platform already holds; it never invents facts; and a person approves every summary before it goes out. A 2025 study in Nature Communications confirmed this design: such tools save time but must support experts, not replace them.

The rule over everything: the AI recommends — SGW's people decide. No AI ability in this product ever takes an action by itself.

Ideas from the case we deliberately did NOT use (yet) — and why. Analysing drone photos of damage (computer vision): SGW has no confirmed drone photo process today, so there is no decision this would feed, revisit later if photos exist. Automatic route planning for crews (optimisation): real value, but it only makes sense after operators trust the risk ranking, it is sequenced for a later phase, not rejected forever.

Case requirement (verbatim): “Outline system components and integration with GIS, weather, and field ops tools.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. The architecture was not designed from taste. It follows from what the abilities in Section 5 need: data must come in, be stored in one place, feed the AI, reach a server, and appear on screens for the four users — that gives five layers. The systems named in the case (asset maps, maintenance, weather, field tools) are fixed integration points; for each, we plan a normal connection and a fallback if no modern connection exists (assumption A1). One safety boundary rules everything: the platform only READS from SGW’s operational systems — it can never send commands to the grid or the water network.



How to read it. Data flows top to bottom: SGW’s four existing systems are only sources. Layer 1 collects. Layer 2 stores one joined record per asset — with the source and age of every value — plus an audit log of every recommendation and every human decision. Layer 3 is the AI from Section 5: it recommends and never acts. Layer 4 is the application server: sign-in, roles, and the confirm-or-override step. Layer 5 is one simple screen per user. The crew job list is built offline-first: it keeps working when the storm cuts the signal, and syncs when the signal returns.

The named integrations — connection and fallback for each:

SGW system (named in the case)	What we take from it	Normal connection	If that is not possible (fallback)
GIS (asset maps)	Asset locations, types, connections	Standard map-data interface (e.g., ArcGIS API)	Scheduled file export (daily)
Maintenance platform	Work orders, inspections, condition notes	The platform’s data interface (API)	Scheduled file export (daily)
Weather feeds	Forecasts and observations, per grid area	Public weather APIs (e.g., national weather service)	Multiple providers — switch source
Field-ops tools	Crew positions, job status, damage reports	The tool’s data interface (API)	Simple structured form in our own crew view

Two honest notes. First: which connections truly exist at SGW is unknown (assumption A1) — the first project phase tests every one, and the fallbacks above mean no missing connection can stop the project. **Second:** joining four systems means matching records that use different ID codes for the same asset; experience across utilities says some records will not match. The data layer therefore includes a matching step with a human check for the records that fail — this is planned work, not a surprise.



Case requirement (verbatim): “Identify required datasets, assumptions, and data quality considerations.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. The dataset list is not a wish list: it is exactly the union of the “what data goes in?” answers from the three AI abilities in Section 5, plus what the audit log needs. The quality risks are not invented either. I downloaded and inspected real public datasets of the same kinds — US government outage records, official weather station files, a public water-sensor benchmark — and copied their documented, measured problems into this plan. The test dataset built for the prototype deliberately contains every one of these problems, so the design is proven against dirty data, not clean data.

Required datasets — one row per input the abilities need:

Dataset (from SGW)	Must contain	Needed by	If not available (fallback)
Asset records (from the GIS maps)	Location, type, connections of every substation, line, plant, pump	Risk forecast · the shared asset registry	Public infrastructure registries to start (assumption A1)
Maintenance history	Inspections, condition notes, repairs, dates	Risk forecast	Start with asset age and type only; add condition later
Weather forecasts, per grid area	Wind gusts, rain, per ~3 km square, updated several times a day	Risk forecast	Public weather services provide this — low risk
Live sensor readings	Water pressure and flow; grid measurements; updated every few minutes	Early warning	Start with the water side only, where sensors exist
Damage and outage records (past storms)	Which assets failed, when, in which storm	Proving the risk forecast on history (assumption A7)	Expert review + simulated storms instead
Crew and job status	Crew positions, assignments, progress	Dispatch board · summary writer	Simple structured form in our own crew view

Data quality — what goes wrong with this kind of data. We did not guess these problems. We opened real public data files of the same kinds and found each problem inside them. Each row: the problem, the proof it is real, and what our design does about it:

What goes wrong	How we know it is real	What we do about it
The same asset has different names in different systems. The map system says SS-1042. The maintenance system says TX-4471. A computer cannot see these are the same station.	In our test data, built to copy real utility data: 17 of 220 assets could not be matched by computer.	The computer matches what it can. A person checks the small list that fails.
Condition information is old. An inspection may be six years in the past. Maybe the asset got worse since then.	In our test data: inspection dates were between 2 months and 6 years old. This is normal in utilities.	Every number on screen shows its date. Example: “Condition: fair (checked 2019)”. The operator can judge.
Weather stations do not measure strong wind gusts most of the time. The gust column is usually empty.	We opened a real US weather station file: 97% of the gust values were missing.	We do not depend on stations. We use weather forecast maps, which give wind for every 3 km square.
Outage records often have a broken “total customers” number. Without a correct total, percentages are garbage.	In a real US outage file: 83% of rows had zero as the total. Zero is clearly wrong.	Before using any total, we check it against public population numbers.
In big storms, one county sometimes gets its neighbors’ outages written into its record.	Documented in the US national outage data: counties showing more customers out than they have customers.	A simple automatic rule flags impossible numbers (more out than exist).
Repair records are not failure records. Many repairs are routine work, not failures.	Real work orders mix true failures with routine jobs like tree cutting. Teaching AI from them teaches lies.	We build the true failure list together with SGW, from their outage records — not from repair logs.
Public outage data counts by county only. It never says which station failed. But our AI must learn per asset.	The US national outage dataset is county-level by design. Everyone faces this wall.	Only SGW’s own records can teach the AI per asset — see the critical dependency below.

The critical dependency: SGW’s own records of which assets failed in past storms (assumption A7). Public data cannot replace them. If they are good, we prove the risk forecast by replaying history. If they are poor, we fall back to expert review and simulated storms — slower, but workable. Finding this out is a week-one task.

Proof of preparation. The prototype ships with a generated test dataset that mirrors real data — same file shapes, same value ranges, and all seven problems above injected on purpose, with a documentation sheet mapping every file to the real public dataset it copies. Anyone can regenerate it from one script.



Case requirement (verbatim): “Address responsible AI usage, oversight mechanisms, and compliance.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. I followed the US government’s framework for trustworthy AI (NIST AI Risk Management Framework, 2023). It says: before launch, answer four questions in writing — who is responsible, what could go wrong, how do we test it, and who watches it after launch. An empty answer means “not ready”, no matter how good the technology is. The answers below are not new inventions: the possible harms come from the mistake-price analysis (Section 9), the tests come from the proof plans in Section 5, and the watching plan comes from Section 5’s “how do we watch it after launch” row. One rule stands above everything: the AI recommends — SGW’s people decide.

The four questions, answered in writing:

Question	SGW answer
1 · Who is responsible for what the AI recommends?	A named person: the operations director — not the software vendor, not the IT department. A review meeting every month: operations + engineering + compliance together look at how the AI performed.
2 · What could go wrong, and who could get hurt?	Four dangers, each with a built-in protection. (a) Help arrives late at a hospital or water plant → the system can NEVER send crews by itself; a person confirms every move. (b) Poorer neighborhoods get less protection than richer ones → we test the rankings for fairness across areas and income groups. (c) People trust the computer too much and stop thinking → every ranking shows its reasons, so people keep judging. (d) A wrong fact reaches a regulator or the press → a person approves every summary before it leaves.
3 · How do we test it before launch?	Replay five years of past storms and check: did damaged assets rank high? Compare rankings across regions and income groups (fairness). Test the screens with operators under storm-like pressure. A security review, because this is critical infrastructure.
4 · Who watches it after launch?	Every month: compare predictions with reality. Automatic alarms when quality drops — then the model is retrained. A short record card for every model version: what it is for, how good it is, where it is weak. And a documented kill-switch: at any moment, SGW can turn the AI off and work manually — the platform still shows all the data.

Human-in-the-loop — the three human control points: (1) every recommendation must be accepted, changed, or rejected by a person (requirement F4); (2) every false alarm can be dismissed with one click, so control stays cheap (F5); (3) every written summary is approved by a person before it goes out. The AI never acts alone — anywhere.

Security, in plain words. Four protections, and the reason for each:

Protection	What it means	Why it matters
Who gets in	Every user signs in, with a second proof of identity (for example, a code on the phone). Each person sees only what their job needs: the dispatcher sees the dispatch board, the executive sees the summary — nobody sees everything by default.	If one account is stolen, the thief gets one narrow view — not the whole platform.
How data is protected	All data is scrambled (encrypted) so that only the platform can read it — both while it travels over the network and while it sits in storage.	Grid and water data describe critical infrastructure. Stolen readable data would be a map for attackers.
The one-way wall	The platform only READS from the systems that physically control the grid and the water network. It has no technical ability to send commands to them — the connection is built one-way.	Worst case: a hacker fully takes over our platform. Even then, they cannot switch anything off. The platform is a window, never a remote control. This separation is standard utility practice (called IT/OT separation).
Everything is recorded	Every sign-in, every data access, every recommendation, and every human decision is saved with a time stamp (the audit log, requirement F8).	After any incident, SGW can reconstruct exactly what happened — and prove it to a regulator.

Compliance — what must be checked with SGW’s lawyers and regulators. Four rule areas, checked in a named early task with SGW’s legal and compliance people. Assumption A8 — that a human keeping final authority satisfies today’s rules — is confirmed or corrected there.

Rule area	In plain words	What it means for this platform
Power-grid security rules (NERC CIP)	Mandatory US rules that protect the computer systems controlling the power grid.	Our one-way wall is designed to keep the platform OUTSIDE the most heavily regulated zone — but this must be verified with the rules, not assumed.
State regulator requirements	Utilities must report outages, response times, and decisions to their state regulator.	Good news: the summary writer and the audit log make this reporting easier and faster — a benefit, not just a duty.
Privacy rules	Screens may show customer-related information (areas without power, critical customers).	Show only what the job needs; show neighborhood totals instead of single homes wherever possible.
Rules for AI itself	Rules for AI in critical services are new and still forming.	Our design follows today’s accepted line: the AI only recommends, a person always decides, and everything is recorded. The compliance check watches for new rules.

Success Metrics, MVP Scope & Delivery Priorities



Case requirement (verbatim): “Define what success looks like and what ships first.”

HOW I PRODUCED THIS SECTION — METHOD AND REASONING. The metrics come from pricing the two possible mistakes (method: cost-sensitive learning, C. Elkan, 2001). Missing a real failure can cost millions; a false alarm costs a wasted trip. Because failures are rare, a model that always says “safe” would be 99% correct — and would miss every real failure. So the usual metric, accuracy, lies here. Our headline metric is instead: the share of real failures the system flagged in advance. The MVP scope comes from Section 2: the first thing we ship is the cheapest thing that tests the three dangerous, unproven guesses (A1: can we reach the data, A2: does a combined view change the decision, A3: will operators act on it). Everything else waits in a ladder, and each rung has a written exit test that unlocks the next.

What success looks like — measured, not felt. All numbers are starting targets, to be set properly with SGW against today's baseline:

Success measure	Starting target	Why this measure
Real failures flagged in advance (checked by replaying past storms)	At least 7 of every 10	Missing a failure is the ruinous mistake — this is the headline number
Time to one full damage picture (dispatcher)	From ~30 minutes to under 5	Directly measures the repaired information gap
Time to build or adjust the storm plan (manager)	From hours to under 1 hour — even when the forecast shifts	The forecast-change restart was the worst break in Section 3
Decisions actually change (test of guess A2)	In scenario tests: operators change their plan in at least 1 of 3 storms	If the view never changes a decision, the product has no reason to exist
Operators act on recommendations (test of guess A3)	Most recommendations accepted or dismissed WITH the reasons read	Measures trust — the second dangerous guess
Summary correctness	Zero facts that skipped human approval; corrections per summary going down	A wrong number reaching a regulator is this ability's worst mistake

What ships first — the MVP. One storm scenario, end to end: the joined asset view, the ranked risk list with plain reasons, the planning view and the dispatch board, replaying one historical storm, running on prepared data files, not live connections. That is enough to test A2 and A3 with real operators. **Deliberately NOT in the MVP:** live connections to all four systems, the water early-warning ability, the summary writer, the offline crew app, and route optimisation — good things, but none of them tests a dangerous guess, so none of them may delay the test.

Delivery priorities — a ladder with exit tests:

Priority	What is built	The exit test that unlocks the next rung
P0 — the MVP probe	Joined view + ranked risks with reasons + planning and dispatch screens, on prepared data	Operators change a decision in scenario tests (A2) AND act on the rankings (A3). If not: redesign, do not scale
P1 — make it real	Live connections to the four systems; the record-matching step; water early warning; the summary writer with human approval	Predictions verified against one real storm season; false-alarm dismissals stay cheap; summaries need few corrections
P2 — extend	Offline crew app; field photo capture; smarter models (physics + learning combined); route suggestions	Each extension must name the decision it improves before it is built — same admission rule as Section 5

The scope rule, in one line: the MVP is not a small product — it is the cheapest honest test of the guesses that could kill the project. Ship the test first; scale only what passes.